



Lecture 5

Clean Architecture

Sharif University of Technology
Winter 2026



Course Overview

A Review of Software Architecture

Concepts to Follow Before Diving into Clean Architecture

A Detailed Overview of Clean Architecture

Case Study: How to Design the Architecture of a Business Using Clean Architecture

Talk is Cheap, Show Me the Code: A Clean Architecture Case Study Using Python



Part I **Introduction**

What Is Design and Architecture?



- The word “**architecture**” is often used in the context of something at a high level that is divorced from the lower-level details, whereas “**design**” more often seems to imply structures and decisions at a lower level:
 - Architecture: Shape of the system
 - Design: Implementation details
- *“The goal of software architecture is to minimize the human resources required to build and maintain the required system.”*

Good Design vs Bad Design



- Good Design:

- Requires low effort to build and maintain
- Keeps maintenance effort low throughout the system's lifetime

- Bad Design:

- Requires increasing effort with each release
- Causes productivity to decline over time
- Eventually slows development to the point of stagnation

Fast or Well



- **The only way to go fast, is to go well**
- The developers may think that the answer is to start over from scratch and redesign the whole system—but that's just the Hare talking again!



A Tale of Two Values



- Software provides two values to stakeholders:
 1. Behavior
 - Make machines behave correctly
 - **What stakeholders often focus on**
 2. Architecture
 - Software must be 'soft' - easy to change
 - **What developers must protect**
- **Development Team's Role: The development team must advocate for architecture to be prioritized by management.**

Adapting to Change



- We live in a world of changing requirements. To ensure our software survives unanticipated changes, we must design for adaptability from the start.
- **Make your code as good as it needs to be at the moment, refactor it as requirements change.**



Part II

Concepts to Know Before Using Clean Architecture

SOLID Overview



- Five principles for mid-level design:

- **S - Single Responsibility Principle**

- A module should answer to only one actor—one reason to change.

- **O - Open-Closed Principle**

- Open for extension, closed for modification—add behavior without changing existing code.

- **L - Liskov Substitution Principle**

- Subtypes must be substitutable for their base types without breaking functionality.

- **I - Interface Segregation Principle**

- Clients shouldn't depend on interfaces they don't use—keep interfaces focused.

- **D - Dependency Inversion Principle**

- Depend on abstractions, not concretions—high-level and low-level modules both depend on interfaces.
- This is the foundational principle for Clean Architecture!

Spot the Violations

```
1 class OrderService:
2     def __init__(self):
3         self.db = MySQLConnection()
4
5     def process_order(self, order):
6         # Validate
7         if order.amount <= 0:
8             raise ValueError("Invalid amount")
9
10        # Save to database
11        self.db.execute(
12            f"INSERT INTO orders VALUES ('{order.id}', '{order.amount}')"
13        )
14
15        # Send confirmation email
16        smtp = smtplib.SMTP('smtp.gmail.com')
17        smtp.send(order.email, "Order confirmed!")
18
19        # Generate PDF receipt
20        pdf = PDFGenerator()
21        pdf.generate(order)
22
23        # Log
24        with open('log.txt', 'a') as f:
25            f.write(f"Order {order.id} processed\n")
```

Which SOLID principles are violated here?

The Fixed Version

```
1 # D - Depend on abstractions
2 class Database:
3     def save(self, data): pass
4
5 class EmailService:
6     def send(self, to, message): pass
7
8 class Logger:
9     def log(self, message): pass
10
11 class ReceiptService:
12     def generate(self, order): pass
13
14 class OrderValidator:
15     def validate(self, order):
16         if order.amount <= 0:
17             raise ValueError("Invalid amount")
```

```
1 # S - Single responsibility per class
2 # O - Extend without modifying OrderService
3 # D - Inject dependencies, not hardcode them
4 class OrderService:
5     def __init__(
6         self,
7         db: Database,
8         email: EmailService,
9         logger: Logger,
10        receipt: ReceiptService,
11        validator: OrderValidator
12    ):
13        self.db = db
14        self.email = email
15        self.logger = logger
16        self.receipt = receipt
17        self.validator = validator
18
19    def process_order(self, order):
20        self.validator.validate(order)
21        self.db.save(order)
22        self.email.send(order.email, "Order confirmed!")
23        self.receipt.generate(order)
24        self.logger.log(f"Order {order.id} processed")
```

What Are Components?



- Components = units of deployment
- Examples:
 - JAR files (Java)
 - DLLs (.NET)
 - Shared libraries (Unix)
 - NPM packages (Node.js)
- Characteristics:
 - Smallest deployable entities
 - Can be independently developed
 - Can be independently deployed

Component Cohesion Principles



- Which classes belong in which components?
 - REP: Reuse/Release Equivalence
 - Group classes that are released together and used together
 - CCP: Common Closure Principle
 - Group classes that change for the same reason
 - CRP: Common Reuse Principle
 - Don't put classes together that aren't used together
- These three principles are in tension!
 - Can't satisfy all three simultaneously
 - Early: Focus on CCP (ease of development)
 - Mature: Move toward REP and CRP (reusability)

Component Cohesion Principles



- **CCP** — *Developer's perspective during development*
 - “If I need to change feature X, how many components do I have to touch?”
 - → **Minimize scattered changes**
- **CRP** — *User's perspective when depending on your code*
 - “If I only need feature X, am I forced to take Y and Z too?”
 - → **Minimize unnecessary dependencies**
- **REP** — *Maintainer's perspective when releasing*
 - “Does this component have a coherent purpose worth versioning and tracking?”
 - → **Minimize meaningless releases**

Logging Library Example



- You have: Logger, FileLogger, ConsoleLogger, DatabaseLogger, LogFormatter, DatabaseConnection
 - CCP (minimize scattered changes) - One component:
 - “Logging”
 - CRP (minimize forced dependencies) - Three components:
 - “Logging-Core” (Logger, LogFormatter)
 - “Logging-File” (FileLogger)
 - “Logging-Database” (DatabaseLogger, DatabaseConnection)
 - REP (coherent releases) - Two components:
 - “Logging-Framework” v1.0 (Logger, FileLogger, ConsoleLogger, LogFormatter)
 - “Logging-Extensions” v1.0 (DatabaseLogger, DatabaseConnection)

Component Coupling Principles



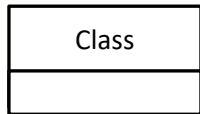
● How should components relate?

- ADP: Acyclic Dependencies Principle
 - Dependency graph must be a DAG (No cycles allowed)
 - Breaking Cycles:
 - Dependency Inversion Principle
 - Create new component for shared code
- SDP: Stable Dependencies Principle
 - Depend in direction of stability
- SAP: Stable Abstractions Principle
 - Stable components should be abstract
 - Unstable components should be concrete

SOLID Principles vs Package Design Principles

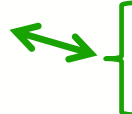


SOLID Principles

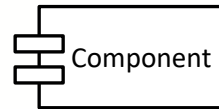


Applied at Module Level

- SRP** The Single Responsibility Principle
- OCp** The Open-Closed Principle
- LSP** The Liskov Substitution Principle
- ISP** The Interface Segregation Principle
- DIP** The Dependency Inversion Principle



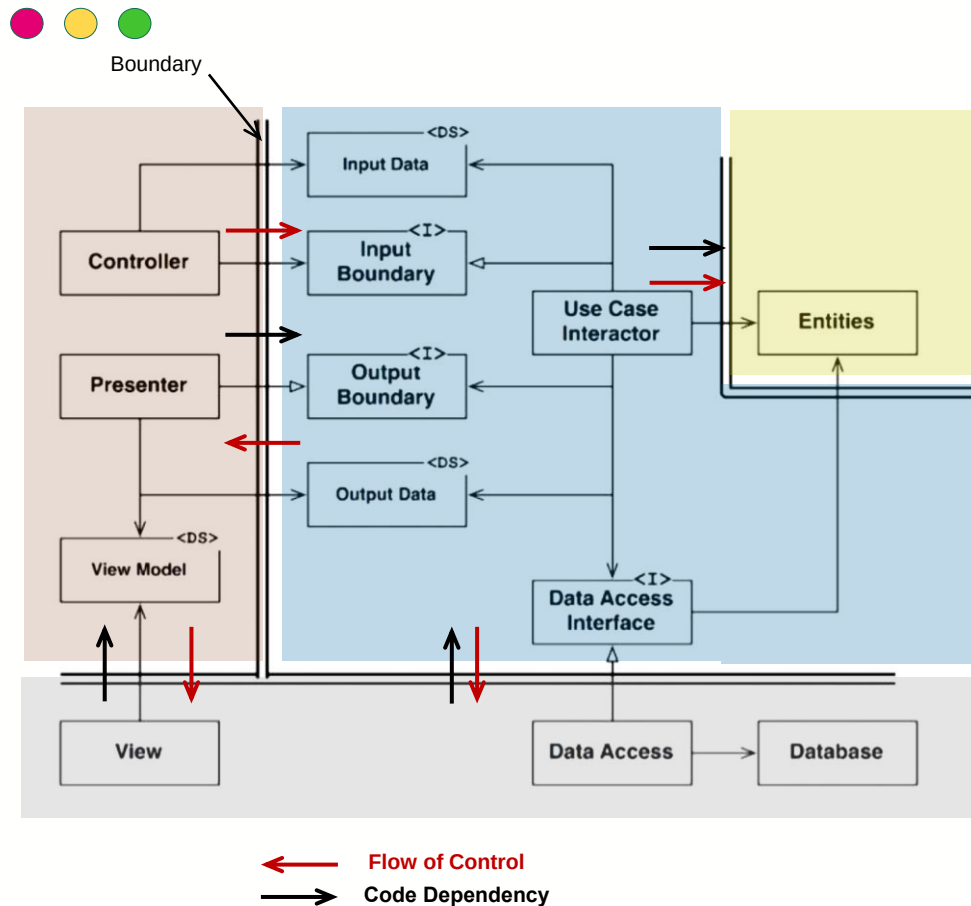
Package Design Principles



Applied at Component Level

- REP** The Reuse-Release Equivalency Principle
- ADP** The Acyclic Dependency Principle
- CCP** The Common Closure Principle
- CRP** The Common Reuse Principle
- SDP** The Stable Dependencies Principle
- SAP** The Stable Abstraction Principle

Architectural Boundaries



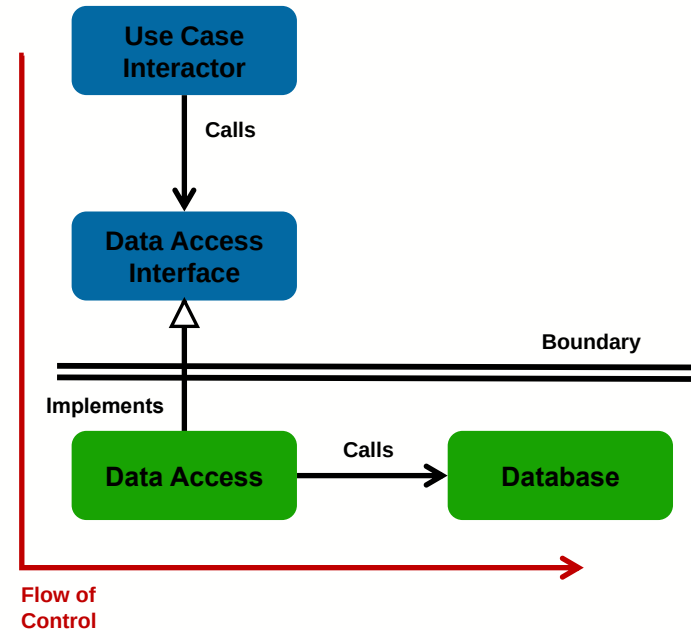
- Boundaries separate different layers and parts of the system
- Boundaries = Layer Separation: Keeps layers independent and decoupled.
- **Dependency Rule: Inner layers should never depend on outer layers.**

Drawing Lines in Software Architecture



● Drawing Boundaries

- Architecture is about drawing **boundaries** between different software elements to **separate concerns** and **limit dependencies**.
- These boundaries help prevent **fragility** in the system, ensuring that **changes in one part** do not **break unrelated parts**.



Drawing Lines in Software Architecture (cont.)



- **Effective Boundaries**

- Boundaries are often drawn where **changes happen at different rates**. For example, **UI changes** happen more frequently than **business logic changes**, so a clear boundary between these two allows changes in the UI to occur without affecting core business rules.

Key Points About Boundaries



● Decoupling:

- By defining clear boundaries, we can **decouple** systems from one another. For example, the **use case layer** (business logic) should not be tightly coupled to the **UI** or **database**.

● Separation of Concerns:

- Boundaries ensure that **different components** or layers (e.g., business logic, UI, database) focus on specific tasks and do not interfere with each other.

● Control of Data Flow:

- A boundary determines how **data** or **control** flows between layers. It ensures that higher-level business logic doesn't depend on external technologies or frameworks.

● Testability:

- Boundaries improve **testability** by allowing developers to isolate the parts of the system for unit testing. For instance, mock objects can replace dependencies across boundaries for testing purposes.

Policy and Level in Software Architecture



What is Policy?

In software, **policy** represents all the business rules and procedures that give the software its value.

Examples:

- How business calculations are performed
- How reports are formatted
- How input data is validated

What is Level?

"The distance from the inputs and outputs"

- **Highest level:** Farthest from I/O
- **Lowest level:** Managing I/O directly

Policy and Level in Software Architecture



- Higher-level policies are farther from the input/output and deal with **more abstract** concerns.
- Lower-level policies handle the details of data input/output and communication with external systems.
- **The Role of Dependencies:**
 - A well-designed system will have **dependencies that point inward**, with low-level components depending on high-level components but not vice versa.
 - This ensures that changes in low-level policies (e.g., database technology) do not affect higher-level business rules.

Example: Encryption Program (Wrong)



The Problem with This Approach

A simple encryption program reads characters from input, translates them, and writes to output. Here's the WRONG way to structure it:

```
function encrypt() {  
  while(true)  
    writeChar(translate(readChar()));  
}
```

Why This Is Wrong:

- ✗ High-level encrypt() depends on low-level I/O
- ✗ Cannot reuse encryption logic elsewhere
- ✗ I/O changes force encryption code changes

Dependency Flow:



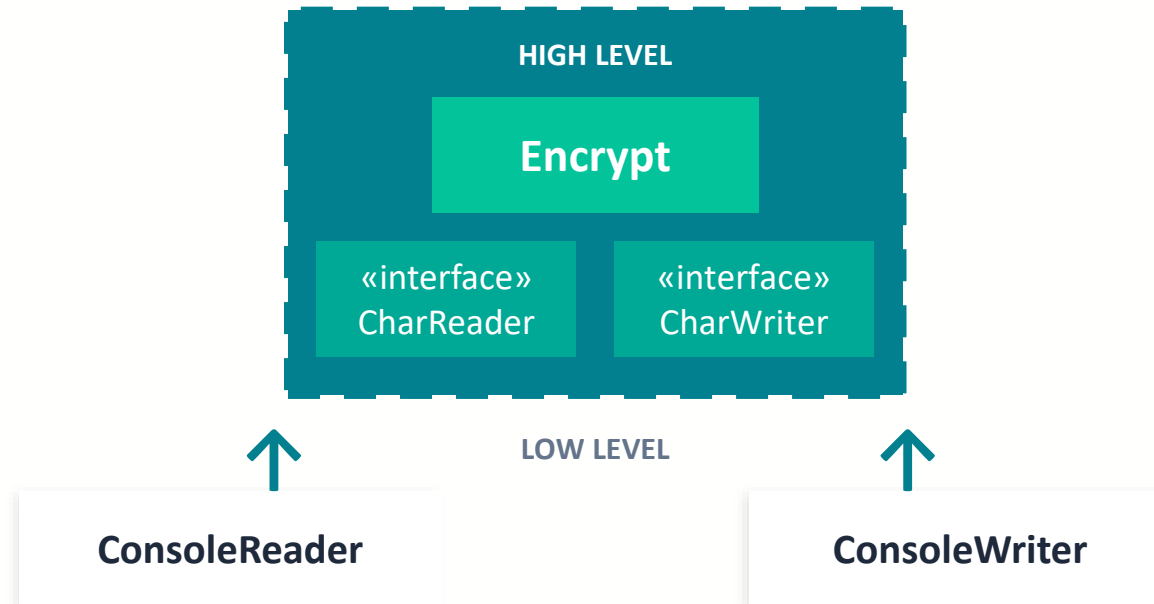
High-level → Low-level (WRONG!)

Example: Encryption Program (Correct)



The Right Architecture

Use interfaces to invert dependencies. The high-level Encrypt component defines what it needs, and low-level I/O implements those interfaces.



Benefits & Key Takeaways



Why This Architecture Matters

Reusability & Flexibility

Encryption logic works with files, network, memory, or any I/O device

Testability

Test encryption with mock I/O - no real console or files needed

Change Isolation

Switching from console to network I/O doesn't touch encryption code

Stability of High-Level Policy

Core business logic remains stable while I/O details can change freely

Remember: The Plugin Architecture

Low-level components (I/O, databases, UI) should plug into high-level components (business logic). Never the other way around. This keeps your most important code independent and stable.

What Are Business Rules?



- Business rules are rules or procedures that make or save the business money. They would make or save money even if executed manually, without any computer system.

- **Two Types of Business Rules**

Critical Business Rules (Entities)

- Core rules critical to the business itself
- Would exist even without automation
- Pure business logic, independent of system

Example: A bank charging N% interest on a loan

Application-Specific Rules (Use Cases)

- Define how automated system operates
- Make sense only in automated context
- Orchestrate entities to achieve goals

Example: Validate credit score before showing loan estimate

Entities: Critical Business Rules



- Entities encapsulate **critical business data** and the rules that operate on that data.
- These should be **independent** of external concerns like the user interface or database. They can be used in any system, regardless of the system's presentation or storage mechanisms.
- The Entity is pure business and nothing else.

Example: Loan Entity

Loan

Critical Business Data:

- principal: Amount
- rate: Percentage
- period: Duration

Critical Business Rules:

- makePayment()
- applyInterest()
- chargeLateFee()

Use Cases: Application Business Rules

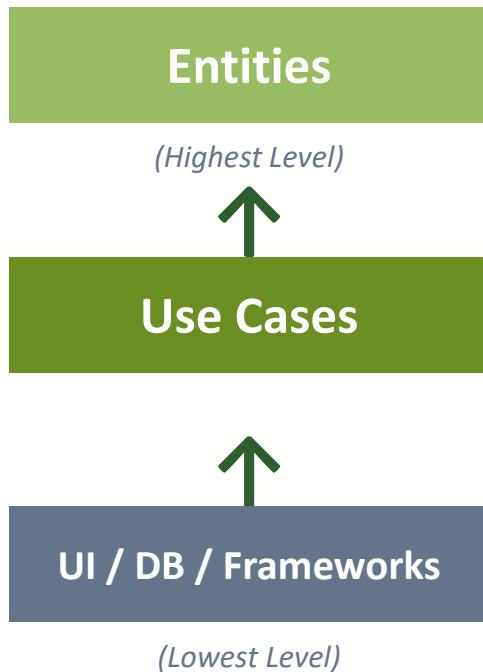


- Use cases are application-specific rules that **manage** how and when entities interact.
- **Example: Gather Info for New Loan**
 - Input: Name, Address, Birthdate, DL#, SSN, Loan Amount
 - Validation: Validate all input • Verify credit score > 500 • Create Customer entity
 - Output: Same info + credit score OR error message if validation fails
- **Key Points About Use Cases**
 - UI-Agnostic: Don't describe how system appears—could be web, mobile, console, or API
 - Orchestrators: Control when and how Entities are invoked
 - Lower Level: Closer to I/O than Entities; use cases depend on Entities, not vice versa

The Big Picture: Why Business Rules Matter



The Dependency Hierarchy



Core Principles

Business Rules Are the Heart

They're why the system exists—the code that makes or saves money

Keep Them Pure

Unsullied by UI, database, or framework concerns

Make Them Testable

Test without web servers, databases, or external dependencies

Plugin Architecture

Everything else plugs into business rules, not the reverse

Screaming Architecture



- The architecture of a software system should scream the **primary business purpose**
- When you examine the **top-level structure** of your application, it should immediately communicate its **core function**



Single-Family Home

You see: Front entrance, foyer, living room, dining room, kitchen, bedrooms...

The architecture SCREAMS: "HOME"



Library Building

You see: Grand entrance, check-in/out desks, reading areas, conference rooms, galleries of bookshelves...

The architecture SCREAMS: "LIBRARY"

The Problem: Framework-Centric Architecture



✗ What Most Projects Look Like

```
my-app/  
├── controllers/  
├── models/  
├── views/  
├── services/  
├── routes/  
└── config/
```

This screams: "WEB FRAMEWORK"

Why This Is Wrong

- New developers learn the framework, not your domain
- Architecture based on tools, not on what the system actually does
- Use cases buried inside framework structures
- Difficult to test without the entire framework running

The Solution: Use-Case Driven Architecture



☒ What Your Architecture Should Look Like

```
healthcare-system/  
├── patients/  
│   ├── RegisterPatient.java  
│   ├── ScheduleAppointment.java  
│   └── UpdateMedicalHistory.java  
├── appointments/  
├── billing/  
├── prescriptions/  
└── entities/
```

This screams: "HEALTH CARE SYSTEM"

Why This Works

- ✓ Developers immediately understand what the system does
- ✓ Use cases are clearly visible and easily found
- ✓ Framework is a detail, not the organizing principle
- ✓ Business logic testable without framework running



Key Principles of Screaming Architecture



1. Use Cases Are Central

Architecture is centered on use cases, not frameworks. Use cases should be the most visible structures in the system.

2. Frameworks Are Tools

Not ways of life. Use frameworks skeptically. Keep them at arm's length to preserve your use-case emphasis.

3. The Web Is a Detail

The web is just an I/O device. Your system should work as a console app, web app, mobile app, or API without fundamental changes.

4. Testable Without Frameworks

Unit-test all use cases without web servers, databases, or frameworks running. Business logic should be testable in isolation.



Part III

The Clean Architecture

The Clean Architecture



- Goal:

- The Clean Architecture divides the system into clear, independent layers that separate the business logic from external elements like UI and frameworks

- Clean Systems are:

- Independent of Frameworks
- Independent of the UI
- Independent of the Database
- Testable

Benefits of Clean Architecture



- Decoupling Layers
 - Ensure that each layer (e.g., UI, business logic, database) is independent to allow changes without affecting others.
- Independent Development & Deployment
 - Teams can work and deploy different parts of the system independently, increasing flexibility and reducing risks.
- Benefits
 - Independent maintainability and scalability.
 - Easier to adapt to changes without impacting the entire system.

Decoupling Modes

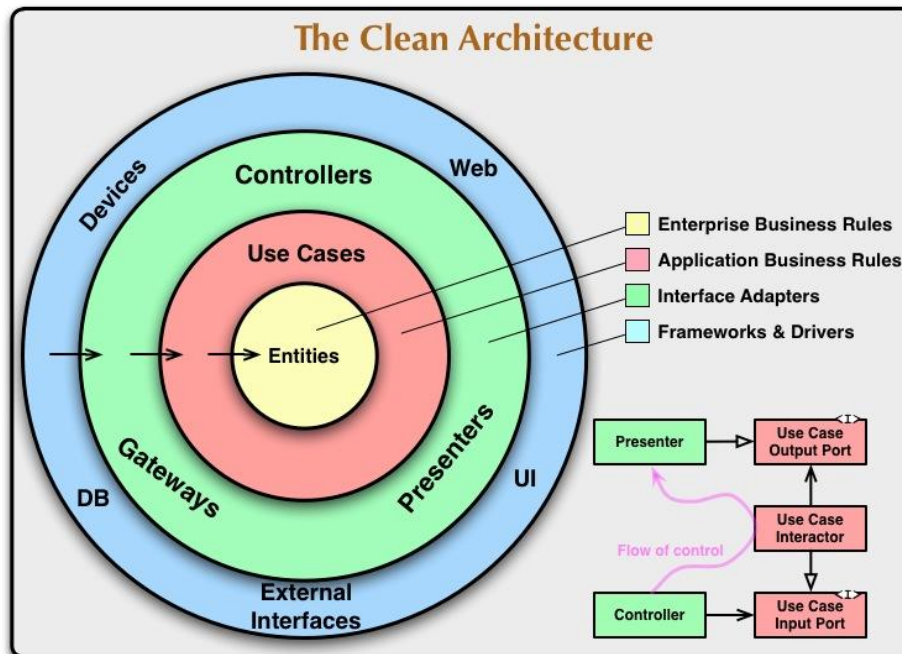


Three levels of decoupling:

- Source Level
 - Monolithic structure
 - Changes don't affect other modules
- Deployment Level
 - JAR files, DLLs, shared libraries
 - Deploy independently
- Service Level
 - Separate processes, network
 - Highest independence

Start with source level, evolve as needed!

The Clean Architecture Diagram



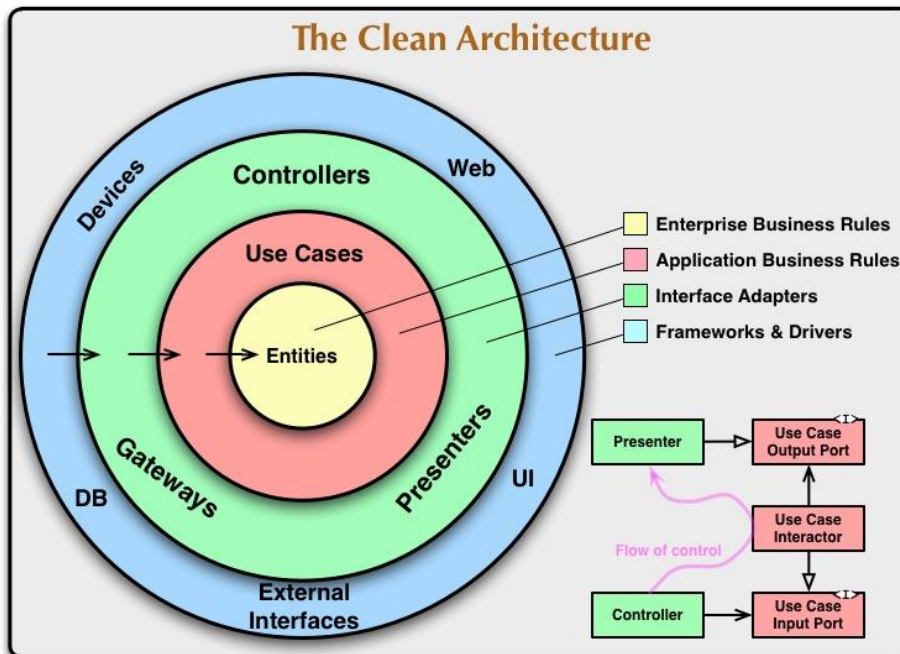
Four Concentric Circles:

1. Entities - Enterprise Business Rules
2. Use Cases - Application Business Rules
3. Interface Adapters - Controllers, Gateways
4. Frameworks & Drivers - UI, DB, Web

The Dependency Rule:

- Dependencies point INWARD only!

The Dependency Rule

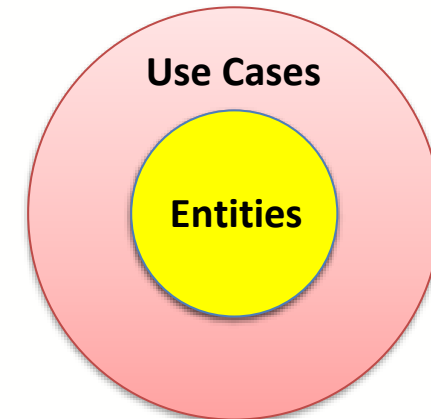


- The **Dependency Rule** states that **source code dependencies must always point inward**, from external components (UI, databases, frameworks) to the core business logic.
- This means:
 - Inner circles don't know about outer
 - Names from outer circles not mentioned in inner
 - Data formats declared in inner circles

Business Rules



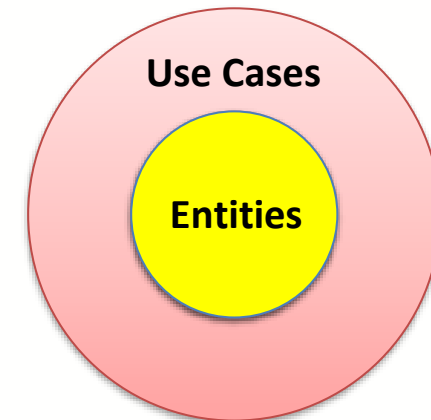
- Business Rules are domain-specific rules and procedures that make or save the business money
- **Entities**
 - Business Rules + Business Data
- **Use Cases**
 - contain rules that specify how and when Business Rules are applied within a software system
 - Use Cases are application-specific
 - **Use Cases orchestrate interactions between Users and Entities**



Business Rules



- Business Rules are domain-specific rules and procedures that make or save the business money
- **Entities**
 - Business Rules + Business Data
- **Use Cases**
 - contain rules that specify how and when Business Rules are applied within a software system
 - Use Cases are application-specific
 - **Use Cases orchestrate interactions between Users and Entities**



Example: Food Delivery App



What are the Entities in a Food Delivery App?



Hint: An Entity = Business Rules + Business Data. Ask yourself: "What things does the app need to know about and apply rules to?"



The Scenario

Customer opens the app and searches for a burger restaurant. He sees a menu, picks 2 items, and places an order. The restaurant confirms it, a driver picks it up, and customer tracks the delivery in real time. Customer pays online and later leaves a 5-star rating.

Example: Food Delivery App (cont.)



Customer

Data: name, address, payment info

Rule: Must be verified before placing an order



Restaurant

Data: name, location, menu, hours

Rule: Can only accept orders during opening hours



Menu Item

Data: name, price, category, stock

Rule: Cannot be ordered if marked unavailable



Order

Data: items, total, status, time

Rule: Total must be above the minimum order value



Driver

Data: name, location, status, rating

Rule: Only available drivers can be assigned a delivery



Delivery

Data: pickup, dropoff, ETA, status

Rule: Status must follow: assigned → picked up → delivered

Example: Food Delivery App (cont.)



Ask the class:

Q Who triggers this use case?

→  The Customer

Q Which entities does it touch?

→ Customer · Restaurant · Menu Item · Order

Q What rule must be checked first?

→ Is the Customer verified?

Q What rule applies to the restaurant?

→ Is it currently open?

Q What rule applies to each item?

→ Is the item available?

Q What rule applies to the total?

→ Is total $\geq$ minimum order?

Use Case: Place Order — Full Flow

1

Customer: Selects restaurant & adds items to cart

touches: **Customer**, **Menu Item**

2

System: Checks: Customer verified? Restaurant open?

touches: **Customer**, **Restaurant**

3

System: Checks: All items available? Total $\geq$ min?

touches: **Menu Item**, **Order**

4

Customer: Confirms order & pays

touches: **Order**, **Customer**

5

Restaurant: Accepts order, starts preparing

touches: **Order**, **Restaurant**

6

System: Assigns an available driver

touches: **Driver**, **Delivery**

Interface Adapters



This layer converts data between the format most convenient for Use Cases/Entities ↔ the format most convenient for external agents (UI, DB, Web).

Controllers

- Accept input from external world
- Convert it into a format the Use Case understands
- Call the appropriate Use Case / Interactor

Presenters & Views

- Receive output from Use Cases
- Format it into a ViewModel
- View renders the ViewModel to screen (Humble Object)

Gateways

- Define interfaces for database operations
- Decouple Use Cases from SQL / ORM details
- Implement CRUD contracts

Controllers — Accepting Input



1	Receive	HTTP request, CLI args, message queue event...
2	Validate	Check required fields, types, format (not business rules!)
3	Map	Convert raw input into a Use Case Input DTO
4	Invoke	Call the Use Case / Interactor
5	Return	Pass the output DTO to Presenter (or return directly)

⚠ **Controllers must NOT contain business logic — that belongs in Use Cases!**

Humble Object Pattern



Some code is hard to test because it's tightly coupled to its environment!



DB Dependency

Cannot run without a real database



UI Coupling

Opens dialogs during logic execution



Side Effects

Sends real emails on every test run



```
1 class OrderProcessor:
2     def process(self):
3         order = db.query("SELECT * FROM orders WHERE status='pending'")
4         if order:
5             messagebox.showinfo("Order", f"Processing: {order.id}")
6             send_email(order.customer_email, "Your order is ready")
```

Humble Object Pattern (cont.)



Definition

Split a class that is hard to test into two parts:

Humble Object: Thin shell that talks to the environment (UI, DB, network). Almost no logic — **too humble to test**.

Testable Core: Pure logic with no environment dependencies. Easy to unit test.

Refactoring to Humble Object

✗ BEFORE — hard to test

```
1 class ReportSender:
2     def send(self, user_id):
3         # Hits the DB directly
4         user = db.get_user(user_id)
5         orders = db.get_orders(user_id)
6
7         # Business logic entangled with IO
8         total = sum(o.amount for o in orders)
9         msg = f"Dear {user.name}, total: {total}"
10
11        # Side effects – can't assert without real SMTP
12        smtp_client.send(user.email, msg)
```



✓ AFTER — Humble Object

```
1 # Testable Core – pure logic, no IO
2 class ReportBuilder:
3     def build(self, name, orders):
4         total = sum(o.amount for o in orders)
5         return f"Dear {name}, total: {total}"
6
7 # Humble Object – thin adapter to infrastructure
8 class ReportSender:
9     def __init__(self, db, smtp, builder):
10         self.db, self.smtp = db, smtp
11         self.builder = builder
12
13     def send(self, user_id):
14         user = self.db.get_user(user_id)
15         orders = self.db.get_orders(user_id)
16         msg = self.builder.build(user.name, orders)
17         self.smtp.send(user.email, msg)
```

Details: Frameworks & Drivers

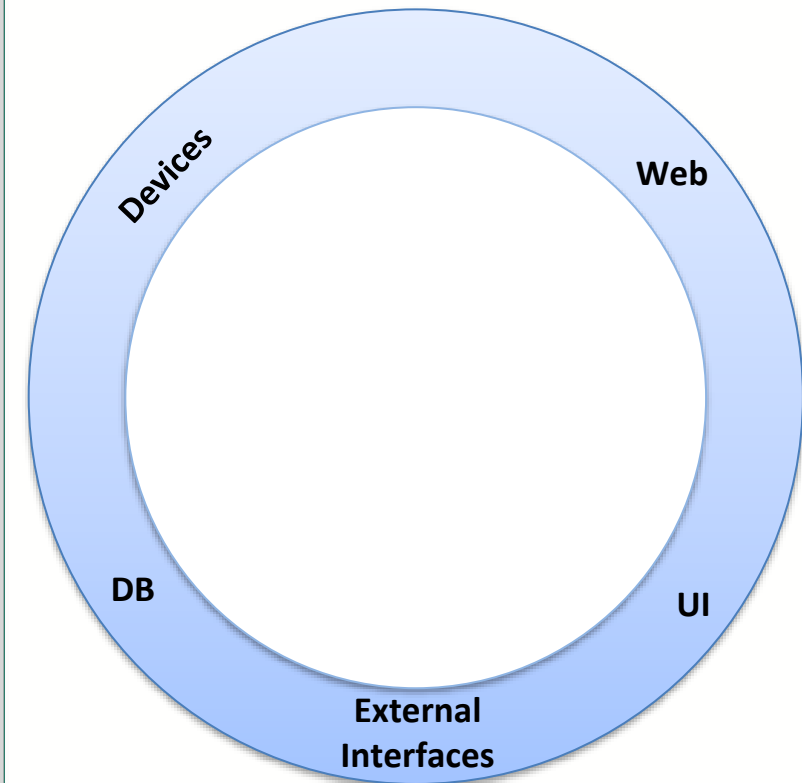


Details only: This layer contains NO business logic

Pluggable: You should be able to swap Postgres for MongoDB without changing Use Cases.

Adapter-mediated: Interface Adapters act as a membrane; Frameworks never call into Use Cases directly.

Test Isolation: You can test all inner layers without a running database or framework.



Data Mappers — Bridging DB and Domain



Objects represent BEHAVIOR, not data. Data Mappers translate between DB rows and Domain Objects.

Domain Object (Entity)

Has methods (behaviors), enforces invariants, no DB awareness

Mapper

DB Record (Table Row)

Flat columns, foreign keys, no methods — just raw data

Presenters & Views



Use Case

(Output DTO)

Presenter

(formats data)

ViewModel

(plain data)

View

(Humble Object)

Presenter

- Receives output DTO from Use Case
- Formats currency, dates, labels
- Builds a plain ViewModel (strings, bools)
- Contains NO rendering logic
- Easy to unit-test — just pure data transformation

View (Humble Object)

- Receives the ViewModel (not business data!)
- Hard to test → kept extremely thin
- Only renders what it's told — no decisions
- Examples: React component
- Rule: 0 business logic inside

Data Mappers (Cont.)

```
1 from domain.order import Order
2 from infrastructure.models import OrderRecord
3
4 class OrderMapper:
5     @staticmethod
6     def to_domain(record: OrderRecord) -> Order:
7         return Order(id=record.id, total=record.total_cents / 100,
8                       status=record.status, items=[...])
9
10    @staticmethod
11    def to_record(order: Order) -> OrderRecord:
12        return OrderRecord(id=order.id, total_cents=int(order.total * 100),
13                           status=order.status)
```

Gateway



A Gateway is an object that encapsulates access to an external resource or system — database, API, file system, message queue — behind a clean interface defined by the use case layer.



Translator

Converts domain objects to/from external data formats (SQL rows, JSON, protobuf)



Boundary Enforcer

Prevents infrastructure details (ORM models, HTTP responses) from leaking into domain logic



Dependency Inverter

Use case defines the interface; gateway implements it — domain never depends on infra

Gateway (Cont.)



The Truth: You Don't Need a Gateway Class

DIP works perfectly fine without one — Layer 2 defines, Layer 4 implements, Layer 3 is optional here!

Uncle Bob's original meaning

Gateway = the abstract interface/port defined in Layer 2

It separates what the use case **needs** from how it is **delivered**.

The concrete implementation can live anywhere outside — Layer 3 or Layer 4 — and still satisfy the boundary.

Common practitioner usage

Gateway = a concrete adapter class placed in Layer 3

This is valid only when Layer 3 has **real work to do**: combining sources, caching logic.

If it does nothing but pass through calls, it's just boilerplate.

Main



Outermost

Main is the last layer — it is just the starting point of your app, not the app itself.

Bootstrapper

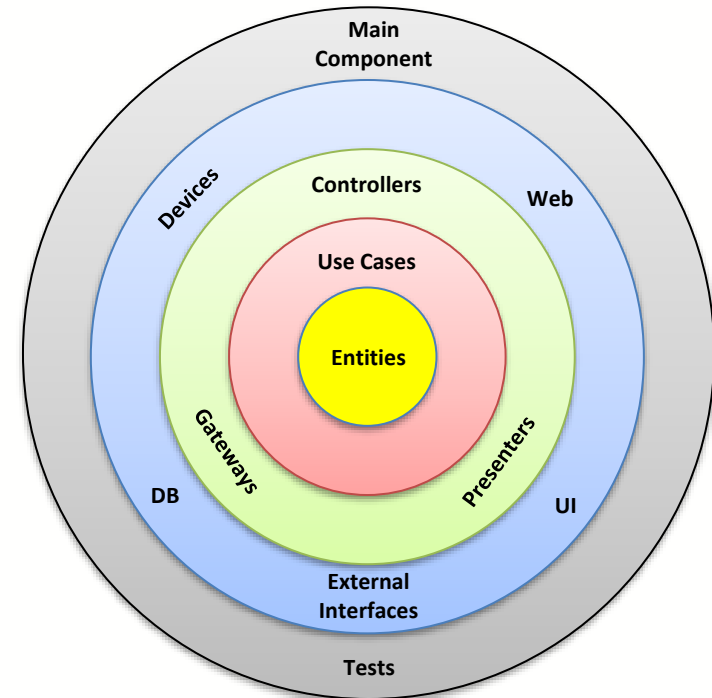
Main creates all objects and connects them together before the app runs.

Dirty by design

Main knows about every layer — and that is okay, because it is the only class allowed to do that.

DI Framework

Instead of Main wiring everything manually, a DI framework does the wiring automatically.



Dependency Injection — Why & How



DI = Don't create your dependencies inside the class. Receive them from outside.



```
1 # ✗ BAD
2 class CreateOrderUseCase:
3     def __init__(self):
4         self.gateway = SQLOrderGateway(SQLAlchemy())
5
6     def execute(self, dto):
7         order = Order.create(dto)
8         self.gateway.save(order)
```



```
1 # ✓ GOOD
2 class CreateOrderUseCase:
3     def __init__(self, gateway: OrderGateway):
4         self.gateway = gateway
5
6     def execute(self, dto):
7         order = Order.create(dto)
8         self.gateway.save(order)
```



Testability

Swap SQLOrderGateway with in-memory repositories in tests — no DB needed



Flexibility

Switch from SQLite to Postgres? Change 1 line in main.py



Clean Rules

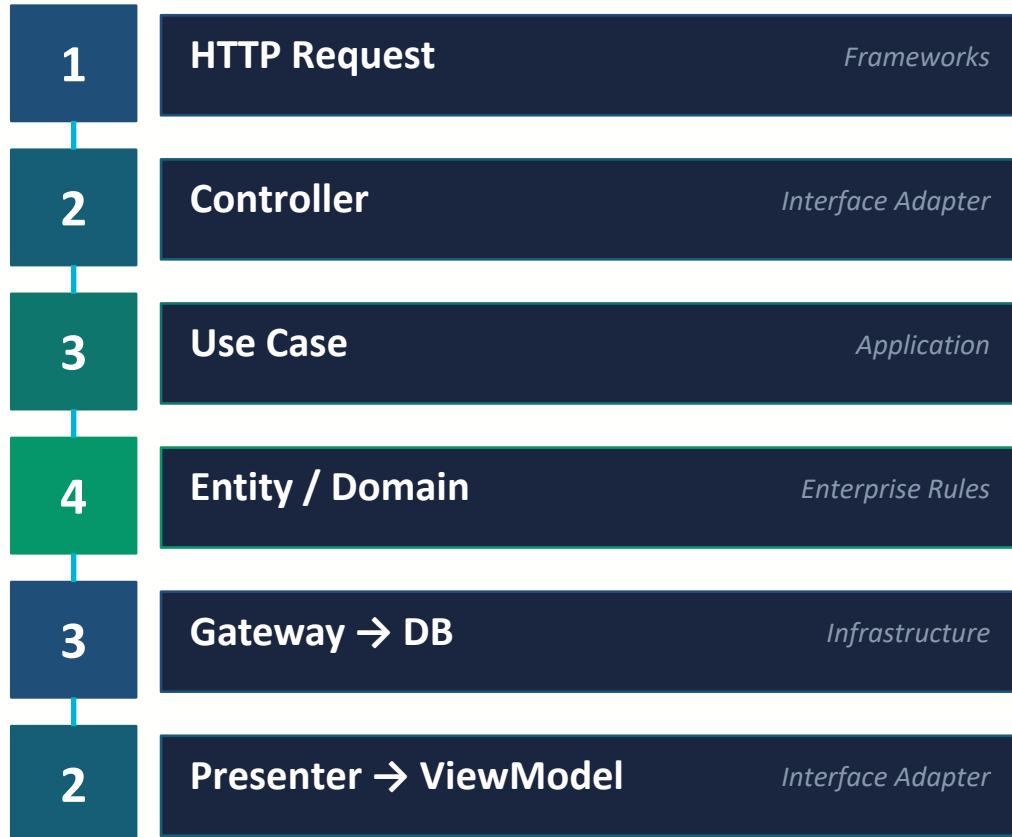
Use Case never imports from infrastructure — dependency rule preserved

Testing Each Layer



Layer	How to Test	Speed
Entities	Pure unit tests — no mocks needed	⚡ Fast
Use Cases	Unit tests with mock gateways (InMemory implementations)	⚡ Fast
Presenters	Unit tests — pure data transformation	⚡ Fast
Controllers	Integration tests with fake use cases	◇ Medium
Gateways	Integration tests against real (test) DB	🐢 Slow
Frameworks/Drivers	End-to-end	🐢 Slowest

A Request Travels Through All Layers



```
# 1. Flask receives HTTP request
POST /orders
{ product_id: 42, quantity: 2 }

# 2. Controller parses & builds DTO
input_dto = CreateOrderInput(42, 2)

# 3. Use Case runs business logic
order = Order.create(input_dto)
# validates stock, calculates price

# 4. Entity enforces invariants
# e.g. quantity must be > 0

# 3b. Gateway persists to DB
gateway.save(order)

# 2b. Presenter formats output
view_model = presenter.present(output_dto)
# → { total_price: '$49.99',
#     status_label: 'Awaiting Shipment' }

# 1b. Framework sends HTTP response
return jsonify(view_model), 201
```

Crossing Boundaries



- At runtime, **crossing boundaries** occurs when a function or operation in one layer calls a function or operation in another layer, passing data between them.
- **Boundary crossings** should be carefully managed to **avoid creating unnecessary dependencies** between components.
- **Source code dependencies** must always point **inward**. This means that the flow of control between layers should respect the **Dependency Rule**: no inner layer should depend on an outer layer
- For example, the core business rules (use cases) should not be aware of or dependent on the UI or database layers.

Crossing Boundaries Using the DIP



Problem: Use Case needs to call Presenter, but Presenter is in an outer layer.

1 Define Interface (Inner Layer)

```
from abc import ABC, abstractmethod

class OutputPort(ABC):
    @abstractmethod
    def present(self, data): ...
```

2 Implement it (Outer Layer)

```
class Presenter(OutputPort):
    def present(self, data):
        # render UI
        print(data)
```

3 UseCase calls through interface

```
class UseCase:
    def __init__(self,
                 out: OutputPort):
        self.out = out
    def execute(self):
        self.out.present(result)
```

Dynamic Polymorphism — The Mechanism



Problem: Use Case needs to call Presenter, but Presenter is in an outer layer.

How it works at runtime:

1. UseCase holds a reference to OutputPort interface
2. At startup, Main injects a concrete Presenter
3. UseCase calls `outputPort.present(data)`
4. At runtime → actual Presenter method executes
5. UseCase has ZERO knowledge of Presenter class

Key Insight

Flow of control crosses the boundary in one direction...

but source code dependency points in the OPPOSITE direction!

This is what makes Clean Architecture possible.

Database $\neq$ Data Model



✗ Common Misconception

"Design the database first."

The database schema defines the architecture. Entities map 1-to-1 with tables. SQL leaks into business logic.

✓ Clean Architecture Truth

"Design the business rules first."

The database is a low-level detail — a storage mechanism. Your Entities should know nothing about how they're persisted.

The data structure used by your application is NOT the same thing as the database. The database should be a plugin you can swap out.

The Gateway Pattern — Making DB a Plugin

Inner Layer (Use Case) defines:

```
# Inner layer — no DB imports!
from abc import ABC, abstractmethod

class UserRepository(ABC):
    @abstractmethod
    def find_by_id(self, id: str) -> User: ...
    @abstractmethod
    def save(self, user: User) -> None: ...
```

Outer Layer (Database) implements:

```
class MySQLUserRepository(UserRepository):
    def find_by_id(self, id: str) -> User:
        # SQL query here
        ...
```

UseCase

calls →

UserRepo
Interface

implements ↓

MySQLRepo

Benefits

- ✓ Swap MySQL → MongoDB with zero business rule changes
- ✓ Test with in-memory fake — no real database needed
- ✓ Database schema can evolve independently

But What About Performance?



A common objection — and a reasonable one.

Performance is important

Query optimization, indexing, caching — these are real concerns.

But they are IMPLEMENTATION details, not architectural decisions.

Architecture first

Get the architecture right first. Performance concerns belong inside the database layer — not in your use cases.

The architecture keeps them contained there.

You can always optimise

If you kept the DB behind a gateway interface, you can add caching, change queries, even change databases...

...without touching business logic!

The Web Is Just an I/O Device



"The Web is an I/O device. Like a terminal from the 1950s — just a different form factor."

What your business rules should NOT know

- ✗ HTTP verbs (GET, POST, PUT...)
- ✗ REST conventions
- ✗ HTML / CSS / JavaScript
- ✗ JSON serialization format
- ✗ Client-Server architecture
- ✗ WebSocket vs HTTP/2

What your business rules should know

- ✓ Domain Entities (User, Order, Product)
- ✓ Business rules and policies
- ✓ InputData / OutputData DTOs
- ✓ Use case interfaces
- ✓ Domain events and errors

Real Example — Payment Processing

Scenario: A payment system calculates fees, processes transactions, handles errors.

✓ Core Business Logic (knows NOTHING about web)

`calculateFee(amount, currency): Money | processTransaction(tx: Transaction): Result | validateCard(card: Card): ValidationResult | handleError(error: DomainError): void`

🔗 Interface Adapters (translate web ↔ core)

`Parse JSON body → Transaction DTO | HTTP 200/400/500 from Result type | Log error to monitoring service
| Serialize Money → { amount, currency }`

Frameworks ≠ Architecture



What frameworks actually are:

- Tools to solve specific, common problems
- Written by people solving THEIR problems
- Opinionated — they want to own your app structure
- Subject to deprecation, breaking changes, abandonment
- Not designed with YOUR business rules in mind

A framework is a tool. A tool should work for you — not the other way around.

The Asymmetric Marriage



The Problem: The Asymmetric Marriage

- You →** Commit to the framework: your whole app is built around it
- Framework →** Doesn't commit to you: breaks your code on v2, v3, v4...
- You →** Spend weeks migrating every major version
- Framework →** Doesn't notice or care

The Solution: Don't Marry the Framework

- ✓ Keep framework code in the outermost layer (Frameworks & Drivers)
- ✓ Never let framework types or annotations leak into your Entities or Use Cases
- ✓ Use Anti-Corruption Layers (adapters) to translate
- ✓ Write business logic as pure classes/functions — no framework imports
- ✓ Treat the framework like a third-party library, not a god object

Framework Coupling — Bad vs Good

✗ Tightly Coupled

Entity knows about Django!
from django.db import models

```
class Order(models.Model):  
    id = models.AutoField(...)
    total = models.DecimalField(...)
```

```
def save(self):  
    # Django ORM baked in!  
    super().save()
```

Use Case imports Flask!
from flask import request, jsonify

```
@app.route('/orders', methods=['POST'])  
def create_order():  
    data = request.json  
    ...
```

✓ Decoupled

Pure Entity — zero framework
class Order:
 def __init__(self, id, total):
 self.id = id
 self.total = total

```
def apply_discount(self, pct):  
    self.total *= (1 - pct)
```

Use Case — no framework imports
class CreateOrderUseCase:
 def __init__(self, repo: OrderRepo):
 self.repo = repo
 def execute(self, data: DTO):
 o = Order(data.id, data.total)
 self.repo.save(o)

Practical Framework Strategy



Rule 1

Keep framework in outer layer

Route handlers, middleware, ORM models — all in the Frameworks & Drivers layer. Nothing bleeds inward.

Rule 2

Use Abstract Interfaces

Your use case depends on IRepository, ILogger, IMailer — not on SQLAlchemy, Winston, or Nodemailer.

Rule 3

Validate at the boundary

Parse and validate framework-specific input (HTTP request) before passing clean DTOs to business logic.

Rule 4

Test without the framework

If you can't unit-test your use case by instantiating a plain class — your architecture is already compromised.

Common Anti-Patterns to Avoid



✗ Business logic in Controllers

```
# ✗ if order.total > 500: apply_discount() # in controller  
# ✓ let the use case handle it
```

✓ Move decision-making to Use Cases / Entities

✗ Use Case imports SQLAlchemy directly

```
# ✗ from sqlalchemy.orm import Session  
# ✓ from use_cases.interfaces import OrderGateway
```

✓ Define a Gateway interface; inject an implementation

✗ Presenter formats data inside the View

```
# ✗ <p>{{ '${:.2f}'.format(order.total) }}</p> # in template  
# ✓ <p>{{ order.total_price }}</p> # already formatted
```

✓ Presenter builds ViewModel first; View only renders it

✗ Entity knows about the database

```
# ✗ class Order: def save(self): db.session.add(self)  
# ✓ class Order: (only domain methods — no DB)
```

✓ Entity has zero awareness of persistence



Part IV

Case Study - Video Sales Platform

What We'll Cover

**1****The Business Idea**

What are we building? Who uses it?

2**Identifying Actors**

Who touches the system? Why does it matter?

3**Defining Use Cases**

What must the system do?

4**Mapping to Layers**

Entities → Use Cases → Adapters →
Frameworks

5**Component Architecture**

Partitioning by actor (SRP in action)

6**Dependency Rule in Practice**

Flow of control vs. flow of source
dependencies

7**The Full Picture**

Putting it all together — the final architecture

Step 1 — The Business Idea



The Product

A website that sells educational video tutorials.

Videos can be streamed online or downloaded permanently for offline use.

Business Model

Individual license

Stream + permanent download

Business license

Streaming only, bulk pricing

Stream Videos

Watch without downloading — anytime, anywhere.

Purchase & Download

Pay once, keep forever — offline access.

Bulk Licensing

Companies buy seats with volume discounts.

Author Tools

Creators upload content, metadata & materials.

Step 2 — What Is an Actor?



An Actor is any person or system that interacts with your application to trigger a behavior.

Human Actors

- Individual buyer
- Business user
- Video author
- Administrator

System Actors

- Payment Gateway
- Database
- Email Service
- CDN / Storage

Four Primary Actors



1

Viewer

Watches video content

Browses catalog, streams purchased content. Main goal: frictionless viewing experience.

2

Purchaser

Buys video licenses

Selects videos, pays, manages licenses. May be an individual or a business.

3

Author

Creates & uploads content

Uploads videos, writes descriptions, attaches supplementary files.

4

Administrator

Manages system & pricing

Sets prices, manages video series, oversees all users and licenses.

Use Cases by Actor



Viewer:

- ✓ View catalog as viewer
- ✓ View purchased videos

Purchaser:

- ✓ View catalog as purchaser
- ✓ Purchase individual license
- ✓ Purchase business license

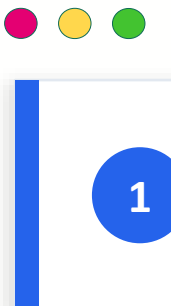
Author:

- ✓ Upload new video
- ✓ Provide video metadata
- ✓ Upload ancillary materials

Administrator:

- ✓ Add/remove video series
- ✓ Set pricing & licenses

Step 3 — Why Start with Use Cases?



1

Use Cases = System Behavior

They describe how the system responds to actors to achieve specific goals. If it's not a use case, it's not a feature.



2

Business-Centric Design

Starting from use cases forces you to think about WHAT the system must do — not HOW it will do it. Technology is secondary.



3

The Heart of Clean Architecture

In Clean Architecture, use cases ARE the application. They are application-specific business rules that sit at the center of your design.

Use Cases by Actor



Viewer	Purchaser	Author	Administrator
View catalog (as viewer)	View catalog (as purchaser)	Upload new video	Add / remove video series
View purchased videos	Purchase individual license	Provide video metadata	Set pricing & licenses
Stream a video	Purchase business license	Upload ancillary materials	Manage all users

Step 4 — Mapping Our Use Cases to Layers



1 Entities

Core business objects. No knowledge of databases, web, or UI.

- Video
- User
- License
- Payment

2 Use Cases

Orchestrate entities to fulfill a business goal. Pure business logic.

- PurchaseVideo
- StreamVideo
- ManageContent
- ManageLicenses

3 Interface Adapters

Translate between use case format and external format (JSON, SQL, etc).

- Controllers (receive HTTP input)
- Presenters (format output for UI)
- Gateways (talk to DB / APIs)

4 Frameworks & Drivers

Details. Swap them without touching business logic.

- Express / Django
- PostgreSQL / MongoDB
- React / Angular
- Payment Gateway SDK

Use Case Deep Dive — Purchase Video



1. User Request

Purchaser — User selects a video and clicks Buy

2. Controller

Interface Adapter — HTTP request arrives;
Controller extracts data and calls use case

3. PurchaseVideo UC

Use Case — Validates user, checks license rules,
delegates payment

4. Entities

Entity Layer — Video, User, Payment objects
enforce business rules

5. Payment Gateway

Framework/Driver — External SDK charges the
user — abstracted behind a Gateway interface

6. Presenter

Interface Adapter — Formats the result
(success/failure) into a View Model

7. View / Response

UI/Delivery — User sees confirmation or error —
no business logic here

Use Case Deep Dive — Stream Video



The Flow

- 1 User logs in → Auth validated
- 2 User selects a video
- 3 StreamVideo use case checks: license valid? not expired?
- 4 Video entity returns CDN URL
- 5 Presenter formats response
- 6 UI renders video player

Business Rules the Use Case Enforces

License Check

Does this user own a license for this video?

License Type

Individual: stream + download.

Business: stream only.

Expiry Check

Is the license still valid?

Format

Return appropriate URL for user's device.

Actor and Use Case Interaction Flow



- **Actor-Driven Flow:**

- **User Request:** An actor (e.g., Individual, Business) triggers a **use case** by requesting an action like purchasing or streaming a video.
- **Use Case Execution:** The system processes the request, interacting with **entities** to carry out business rules (e.g., check payment, validate license).
- **Return Results:** Once the use case is completed, the system returns a result to the **view** via the **controller**, displaying relevant information or updating user records.

- **Example:**

- **Actor:** Individual User → **Use Case:** Purchase Video → **Entities:** Video, User, Payment → **Return Result:** Access granted to video for streaming.

Step 5 — Component Architecture



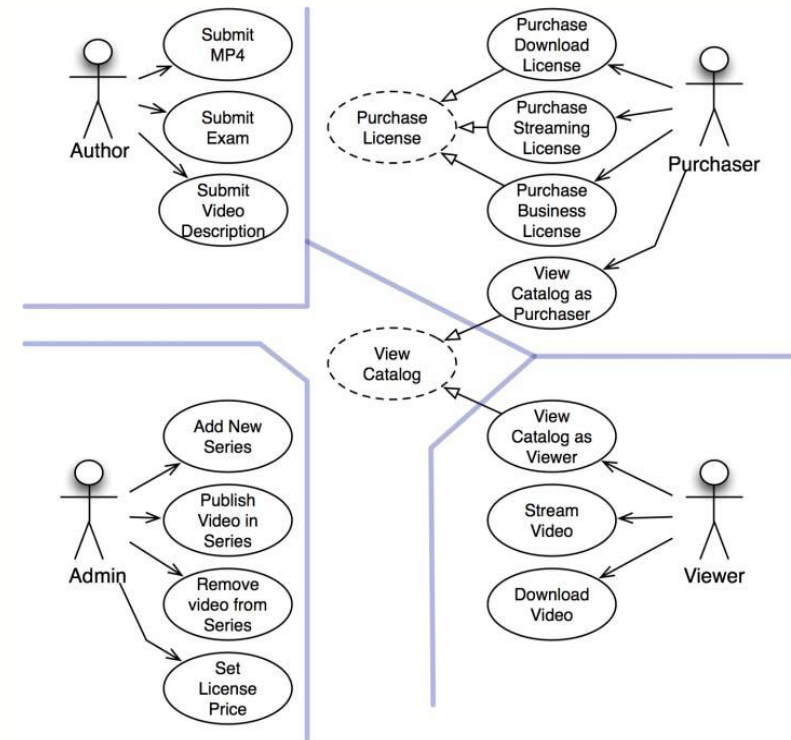
Partition by Actor (SRP)

Each component separated by actor:

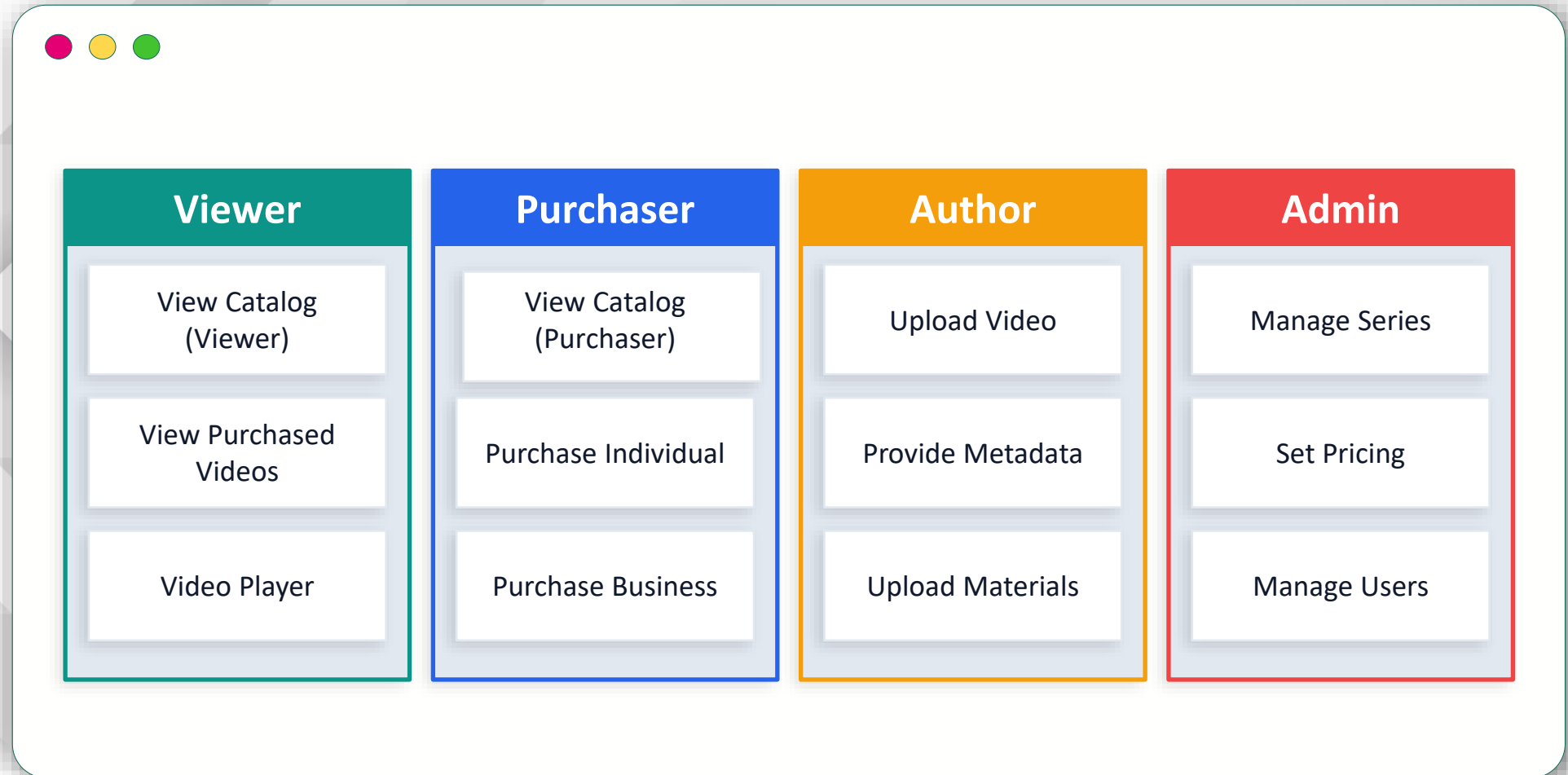
- Viewer Components
- Purchaser Components
- Author Components
- Administrator Components

Within each actor partition:

- Views
- Presenters
- Interactors
- Controllers



Component Architecture (Cont.)



Step 6 — The Dependency Rule in Practice



"Source code dependencies must point only INWARD, toward higher-level policies. Nothing in an inner circle can know anything about an outer circle."

Flow of Control (Runtime)

Controller → Interactor
Interactor → Presenter
Presenter → View

Right to Left at runtime
(this is how data flows)

Source Dependencies (Build Time)

Controller → Interactor ✓
Presenter → Interactor ✓
View → Presenter ✓

Always point INWARD
(toward business rules)

Dependency Inversion — Keeping the DB Outside



Problem: Our Use Case needs to save a purchase. But it can't depend on the database



The Use Case calls an interface — NOT the real payment gateway.

The real adapter (Stripe, PayPal) lives in the outer layer and implements that interface.

Source dependency: Adapter → Interface. The inner layer knows nothing of the outer.

You can swap Stripe for PayPal without touching a single line of business logic.

What Changes — and Why It Matters



Business Decision: Add a rental model (24-hour access)

Must change: Use Cases (new RentVideo UC), Entities (License gets expiry logic)

✓ Stays untouched: Controllers, Views, DB schema, Payment SDK

Tech Decision: Switch from MySQL to MongoDB

Must change: DB Adapter / Gateway implementation only

✓ Stays untouched: ALL business logic — Entities, Use Cases untouched

UI Change: Mobile app instead of website

Must change: Controllers, Views — new delivery mechanism

✓ Stays untouched: ALL business logic — Use Cases, Entities untouched

Payment: Replace Stripe with PayPal

Must change: PaymentGateway Adapter only

✓ Stays untouched: PurchaseVideo Use Case — it only knows the interface

Key Takeaways



1

Always start with the business: Who are the actors? What do they need to do?

2

Use Cases ARE the application. Everything else is a detail.

3

The Dependency Rule: Source code always points inward — never outward.

4

Separate components by actor (SRP). Each actor = its own reason to change.

5

Use interfaces to invert dependencies so inner layers stay pure.

6

Frameworks, databases, and UI are DETAILS. They live on the outside.



CLEAN ARCHITECTURE

**Good architecture
makes details deferrable.**

— *Robert C. Martin (Uncle Bob)*



CLEAN ARCHITECTURE

The Big Picture

What we learned, and how it all fits together.

Clean Architecture Is a Mindset



Separation of Concerns

Business logic must stay independent from databases, UI, and external systems. The core never knows about the outside world.



Leave Options Open

Delay technology decisions as long as possible. Don't commit to specific frameworks or databases until you must.



Start Small, Think Big

Apply SOLID principles and Clean Architecture from day one — even in small projects. Retrofitting boundaries later is very costly.



Grow with Use Cases

As the system grows, use Package Design Principles and organize around formal Use Cases for clarity and scalability.

Why Clean Architecture?



The Problem with Traditional Architectures



Tight Coupling

Business logic mixed with framework code, database queries, and HTTP handling



Hard to Test

Can't test business logic without database, web server, or external services



Framework Lock-in

Switching from FastAPI to Django (or database) requires rewriting everything



Difficult to Maintain

Changes ripple through entire codebase; unclear where business logic lives

Architectural Layers & Boundaries

**5****MAIN**

Wires everything together. Entry point of the app.

*Outermost***4****ENVIRONMENT**

Frameworks, drivers, external tools (DB, Web, Devices).

*Infrastructure***3****ADAPTERS**

Controllers, Gateways, Presenters. Converts data between layers.

*Interface***2****USE CASES**

Application-specific business rules. Orchestrates Entities.

*Application***1****ENTITIES**

Core business rules. Most stable, no dependencies outward.

Innermost

← *Dependency Direction*

How to Create Architectural Boundaries

**01**

One Virtual Folder Per Layer

Each architectural layer gets its own folder. This enforces physical separation and makes boundaries visible in the codebase.

02

Classes Stay in Their Layer

Never mix classes across layers. A Use Case class lives in `/use_cases/`, not in `/adapters/` or `/entities/`.

03

Name Classes for Their Layer

Choose names that signal what role the class plays and which layer it belongs to. Names communicate architecture.

You now have the full map!



Quick Recap

- ✓ Architecture = separating concerns
- ✓ Layers: Entities → Use Cases → Adapters → Env → Main
- ✓ Dependencies always point inward
- ✓ DIP
- ✓ Never add boundaries late — plan ahead

Apply it -> Refactor -> Repeat



Part V

Clean Architecture in Practice

Building Web Applications with FastAPI,
SQLAlchemy & Pydantic

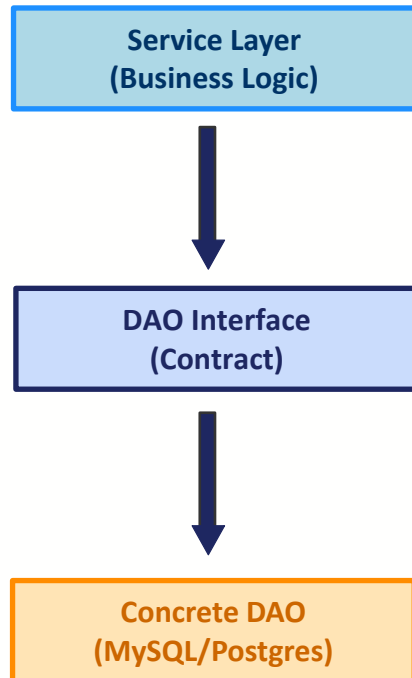


Data Mapper Pattern



```
1 class User:
2     def __init__(self, id: int, name: str, email: str):
3         self.id = id
4         self.name = name
5         self.email = email
6
7 class UserMapper:
8     def __init__(self, db_connection):
9         self.db = db_connection
10
11     def find_by_id(self, user_id: int) -> User:
12         row = self.db.execute("SELECT * FROM users WHERE id = ?", [user_id])
13         return User(row['id'], row['name'], row['email'])
14
15     def save(self, user: User) -> None:
16         self.db.execute("UPDATE users SET name=?, email=? WHERE id=?", [user.name, user.email, user.id])
```

DAO (Data Access Object) Pattern



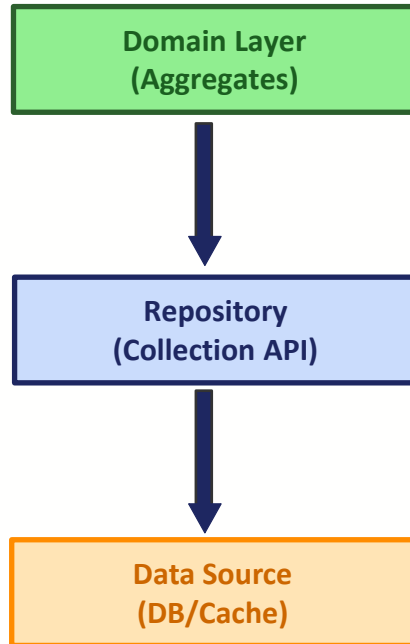
```
1 from abc import ABC, abstractmethod
2 from typing import List, Optional
3
4 class UserDAO(ABC):
5     @abstractmethod
6     def create(self, name: str, email: str) -> int:
7         pass
8
9     @abstractmethod
10    def get_by_id(self, user_id: int) -> Optional[dict]:
11        pass
12
13    @abstractmethod
14    def get_all(self) -> List[dict]:
15        pass
16
17 class MySQLUserDAO(UserDAO):
18     def __init__(self, connection):
19         self.conn = connection
20
21     def create(self, name: str, email: str) -> int:
22         cursor = self.conn.execute(
23             "INSERT INTO users (name, email) VALUES (?, ?)", [name, email])
24         return cursor.lastrowid
```

Repository Pattern



Collection metaphor:

- Work with domain objects as in-memory collections



```
1 from abc import ABC, abstractmethod
2 from typing import List, Optional
3
4 # Domain Model
5 class Order:
6     def __init__(self, id: int, customer_id: int, total: float):
7         self.id = id
8         self.customer_id = customer_id
9         self.total = total
10
11 # Repository Interface
12 class OrderRepository(ABC):
13     @abstractmethod
14     def add(self, order: Order) -> None:
15         pass
16
17     @abstractmethod
18     def find_by_customer(self, customer_id: int) -> List[Order]:
19         pass
20
21     @abstractmethod
22     def find_high_value_orders(self, min_amount: float) -> List[Order]:
23         pass
```

The REAL Differences



Question	Data Mapper	DAO	Repository
What does it return	Domain objects	Dicts/DTOs	Domain objects
Where's the logic	IN Mapper class	IN services	IN domain objects
Query methods	find_by_id(), save()	get(), update(), delete()	find_by_customer() find_expensive()
Main purpose	Isolate domain	Hide DB details	Collection metaphor



Mapper vs Repository

ALMOST identical!

Repository = Mapper + domain queries

Mapper: find_by_id()

Repo: find_by_customer(), find_expired()



DAO is Different

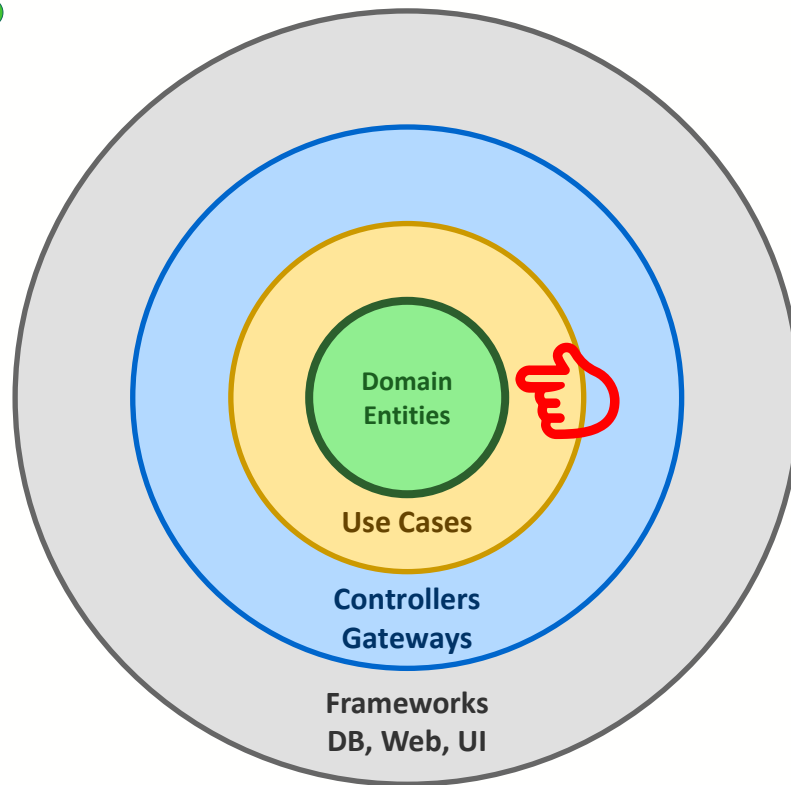
Returns dicts, not domain objects

No business logic in domain

Logic lives in SERVICE layer

Domain logic can still live in domain objects

Wait... What is "Domain"?



Domain = Core Business Logic

In Clean Architecture, this is your "Entities" layer

It contains:

- ✓ Business rules
- ✓ Core models
- ✓ Domain logic
- ✓ Independent of DB/UI

Example:

*User, Order, Product
with their behaviors*

Sample Project: Task Management API



A REST API for managing tasks with the following features:

- Create new tasks with validation
- Retrieve all tasks with filtering

Technology Stack

- | | |
|-------------------------------|-----------------|
| • FastAPI | Web framework |
| • SQLAlchemy | Database ORM |
| • Pydantic | Data validation |
| • Python typing | Type hints |
| • Dependency Injection | DI pattern |

What We'll Demonstrate

- ✓ Separation of concerns
- ✓ Dependency Inversion Principle
- ✓ Testing without infrastructure
- ✓ Clean project structure

Project Directory Structure

```
src/  
├── domain/  
│   ├── entities/  
│   ├── services/  
│   ├── value_objects/  
│   └── exceptions.py  
├── application/  
│   ├── use_cases/  
│   ├── dtos/  
│   ├── interfaces/  
│   └── exceptions.py  
├── presentation/  
│   └── api/  
├── infrastructure/  
│   ├── db/  
│   └── mappers/  
├── config/  
│   └── dependencies.py  
└── app.py
```

Core business logic

Use cases & workflows

API endpoints

Database & external

Layer 1: Domain (Entities)

Pure business logic with NO external dependencies

```
1  @dataclass
2  class Task:
3      id: Optional[int]
4      title: str
5      description: str
6      status: TaskStatus
7      priority: Priority
8      created_at: datetime
9      updated_at: datetime
10     user_id: int
11     due_date: Optional[datetime]
12
13     def mark_as_completed(self) -> None:
14         self.status = TaskStatus.COMPLETED
15         self.updated_at = datetime.now(timezone.utc)
16
17     def is_overdue(self, current_date: datetime) -> bool:
18         if self.due_date and self.status != TaskStatus.COMPLETED:
19             return current_date > self.due_date
20         return False
```

Layer 1: Domain (Value Objects)

Immutable *objects defined by their value, not identity*



```
1 # src/domain/value_objects/task_status.py
2 from enum import Enum
3
4 class TaskStatus(str, Enum):
5     TODO = "todo"
6     IN_PROGRESS = "in_progress"
7     COMPLETED = "completed"
8     CANCELLED = "cancelled"
```



```
1 # src/domain/value_objects/priority.py
2 from enum import Enum
3
4 class Priority(str, Enum):
5     LOW = "low"
6     MEDIUM = "medium"
7     HIGH = "high"
8     CRITICAL = "critical"
```

Layer 2: Application (Interfaces)

Abstract interfaces - Dependency Inversion Principle (DIP)

```
1  # src/application/interfaces/task_repository.py
2  from abc import ABC, abstractmethod
3  from typing import List, Optional
4
5  from ..dtos.task_dto import TaskFilterDTO
6  from ...domain.entities.task import Task
7
8
9  class ITaskRepository(ABC):
10     @abstractmethod
11     async def create(self, task: Task) -> Task:
12         pass
13
14     @abstractmethod
15     async def get_by_id(self, task_id: int, user_id: int) -> Optional[Task]:
16         pass
17
18     @abstractmethod
19     async def get_all_by_user(self, user_id: int, filters: TaskFilterDTO) -> List[Task]:
20         pass
```

Dependency Inversion Principle (DIP)



WRONG (Traditional)

Use Case → SQLAlchemy

Business logic depends on
database implementation



CORRECT (DIP)

Use Case → IRepository ← SQLAlchemy

Both depend on abstraction
Easy to swap implementations



Key Point: Infrastructure (SQLAlchemy) depends on Application (IRepository)

Layer 2: Application (DTOs)

Data Transfer Objects for crossing layer boundaries

```
1 @dataclass
2 class CreateTaskDTO:
3     title: str
4     description: str
5     priority: Priority
6     due_date: Optional[datetime]
7     user_id: int
8
9 @dataclass
10 class TaskFilterDTO:
11     status: Optional[TaskStatus] = None
12     priority: Optional[Priority] = None
13     due_before: Optional[datetime] = None
14     limit: int = 100
15     offset: int = 0
```

```
1 @dataclass
2 class TaskResponseDTO:
3     id: int
4     title: str
5     description: str
6     status: TaskStatus
7     priority: Priority
8     created_at: datetime
9     updated_at: datetime
10     due_date: Optional[datetime] = None
```

Layer 2: Application (Use Cases)

Business workflows orchestrating domain logic

```
1 # src/application/use_cases/create_task.py
2 class CreateTaskUseCase:
3     def __init__(self, task_repository: ITaskRepository):
4         self._repository = task_repository
5
6     async def execute(self, dto: CreateTaskDTO) -> TaskResponseDTO:
7         task = Task.create(
8             title=dto.title,
9             description=dto.description,
10            priority=dto.priority,
11            user_id=dto.user_id,
12            due_date=dto.due_date,
13        )
14
15        created_task = await self._repository.create(task)
16
17        return TaskResponseDTO(
18            id=created_task.id,
19            title=created_task.title,
20            description=created_task.description,
21            status=created_task.status,
22            priority=created_task.priority,
23            created_at=created_task.created_at,
24            updated_at=created_task.updated_at,
25            due_date=created_task.due_date,
26        )
```

```
1 # src/application/use_cases/get_tasks.py
2 class GetTasksUseCase:
3     def __init__(self, task_repository: ITaskRepository):
4         self._repository = task_repository
5
6     async def execute(self, user_id: int, filters: TaskFilterDTO) \
7         -> List[TaskResponseDTO]:
8         tasks = await self._repository.get_all_by_user(
9             user_id=user_id, filters=filters)
10        return [
11            TaskResponseDTO(
12                id=t.id,
13                title=t.title,
14                description=t.description,
15                status=t.status,
16                priority=t.priority,
17                created_at=t.created_at,
18                updated_at=t.updated_at,
19                due_date=t.due_date,
20            )
21            for t in tasks
22        ]
```

Layer 3: Presentation (API Controllers)

FastAPI endpoints that delegate to use cases



```
1  async def create_task(  
2      request: TaskCreateRequest,  
3      use_case: CreateTaskUseCase  
4      = Depends(get_create_task_use_case),  
5      current_user_id: int = Depends(get_current_user),  
6  ) -> TaskResponse:  
7      dto = CreateTaskDTO(  
8          title=request.title,  
9          description=request.description,  
10         priority=request.priority,  
11         due_date=request.due_date,  
12         user_id=current_user_id,  
13     )  
14     result = await use_case.execute(dto)  
15     return TaskResponse.from_dto(result)
```



```
1  @router.get("/", response_model=List[TaskResponse])  
2  async def get_tasks(  
3      status: Optional[TaskStatus] = None,  
4      priority: Optional[Priority] = None,  
5      limit: int = 100,  
6      offset: int = 0,  
7      use_case: GetTasksUseCase = Depends(get_get_tasks_use_case),  
8      current_user_id: int = Depends(get_current_user),  
9  ) -> List[TaskResponse]:  
10     filters = TaskFilterDTO(status=status,  
11                             priority=priority,  
12                             limit=limit,  
13                             offset=offset)  
14     dtos = await use_case.execute(user_id=current_user_id,  
15                                   filters=filters)  
16     return [TaskResponse.from_dto(d) for d in dtos]
```

Layer 3: Presentation (Pydantic Schemas)

NOTE: Pydantic is a Presentation layer tool for validation

```
1 # src/presentation/api/schemas/task_schema.py
2
3 class TaskCreateRequest(BaseModel):
4     model_config = ConfigDict(str_strip_whitespace=True)
5     title: str = Field(..., min_length=1, max_length=200)
6     description: str = Field(..., max_length=2000)
7     priority: Priority
8     due_date: Optional[datetime] = None
```

```
1 class TaskResponse(BaseModel):
2     id: int
3     title: str
4     description: str
5     status: TaskStatus
6     priority: Priority
7     created_at: datetime
8     updated_at: datetime
9     due_date: Optional[datetime] = None
10
11 @classmethod
12 def from_dto(cls, dto: TaskResponseDTO) -> "TaskResponse":
13     return cls(
14         id=dto.id,
15         title=dto.title,
16         description=dto.description,
17         status=dto.status,
18         priority=dto.priority,
19         created_at=dto.created_at,
20         updated_at=dto.updated_at,
21         due_date=dto.due_date,
22     )
```

Layer 4: Infrastructure (Repository)

SQLAlchemy implementation of ITaskRepository interface

```
1 # src/infrastructure/db/task_repository.py
2 class SQLAlchemyTaskRepository(ITaskRepository):
3     def __init__(self, session: AsyncSession):
4         self._session = session
5         self._mapper = TaskMapper()
6
7     async def create(self, task: Task) -> Task:
8         model = self._mapper.to_model(task)
9         self._session.add(model)
10        await self._session.commit()
11        await self._session.refresh(model)
12        return self._mapper.to_entity(model)
13
14    async def get_by_id(self, task_id: int, user_id: int) -> Optional[Task]:
15        stmt = select(TaskModel).where(
16            TaskModel.id == task_id,
17            TaskModel.user_id == user_id
18        )
19        result = await self._session.execute(stmt)
20        model = result.scalar_one_or_none()
21        return self._mapper.to_entity(model) if model else None
22    ...
```

Layer 4: Infrastructure (ORM Models)

SQLAlchemy models - separate from domain entities!

```
1 # src/infrastructure/db/models.py
2 class Base(DeclarativeBase):
3     pass
4
5 class TaskModel(Base):
6     __tablename__ = "tasks"
7
8     id = Column(Integer, primary_key=True, autoincrement=True)
9     title = Column(String(200), nullable=False)
10    description = Column(String(2000), nullable=False)
11    status = Column(SQLEnum(TaskStatus), nullable=False, default=TaskStatus.TODO)
12    priority = Column(SQLEnum(Priority), nullable=False, default=Priority.MEDIUM)
13    created_at = Column(
14        DateTime, nullable=False, default=lambda: datetime.now(timezone.utc)
15    )
16    updated_at = Column(
17        DateTime, nullable=False,
18        default=lambda: datetime.now(timezone.utc),
19        onupdate=lambda: datetime.now(timezone.utc),
20    )
21    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
22    due_date = Column(DateTime, nullable=True)
```

Layer 4: Infrastructure (Mappers)

Model and Model-Entity mappers

```
1  # src/infrastructure/mappers/task_mapper.py
2
3  class TaskMapper:
4      def to_entity(self, model: TaskModel) -> Task:
5          """ORM model -> domain entity."""
6          if model is None:
7              raise ValueError("TaskModel cannot be None")
8          return Task(
9              id=model.id,
10             title=model.title,
11             description=model.description,
12             status=model.status,
13             priority=model.priority,
14             created_at=model.created_at,
15             updated_at=model.updated_at,
16             user_id=model.user_id,
17             due_date=model.due_date,
18         )
19
20     def to_model(self, task: Task) -> TaskModel:
21         """Domain entity -> ORM model."""
22         ...
```

```
22         kwargs = dict(
23             title=task.title,
24             description=task.description,
25             status=task.status,
26             priority=task.priority,
27             created_at=task.created_at,
28             updated_at=task.updated_at,
29             user_id=task.user_id,
30             due_date=task.due_date,
31         )
32         if task.id is not None:
33             kwargs["id"] = task.id
34         return TaskModel(**kwargs)
```

FastAPI Integration with DI

Wire up dependencies and inject them into use cases

```
1  # src/config/dependencies.py
2  async def get_db(request: Request) -> AsyncGenerator[AsyncSession, None]:
3      async with request.app.state.session_factory() as session:
4          try:
5              yield session
6          except Exception:
7              await session.rollback()
8              raise
9
10
11 def get_task_repository(session: AsyncSession = Depends(get_db)) -> ITaskRepository:
12     return SQLAlchemyTaskRepository(session)
13
14
15 def get_create_task_use_case(
16     task_repository: ITaskRepository = Depends(get_task_repository),
17 ) -> CreateTaskUseCase:
18     return CreateTaskUseCase(task_repository=task_repository)
19 ...
```

Testing Benefits of Clean Architecture



```
1 # tests/application/test_create_task_use_case.py
2 async def test_create_task_success(mock_task_repository):
3     """Test use case WITHOUT database, web server,
4         or external dependencies.
5     """
6     mock_task_repository.create.return_value = Task(
7         id=1,
8         title="Test",
9         description="Test task",
10        status=TaskStatus.TODO,
11        priority=Priority.HIGH,
12        created_at=datetime.now(timezone.utc),
13        updated_at=datetime.now(timezone.utc),
14        user_id=1,
15        due_date=None,
16    )
17 ...
```

```
17 use_case = CreateTaskUseCase(task_repository=mock_task_repository)
18
19 dto = CreateTaskDTO(
20     title="Test",
21     description="Test task",
22     priority=Priority.HIGH,
23     due_date=None,
24     user_id=1,
25 )
26 result = await use_case.execute(dto)
27
28 assert result.id == 1
29 assert result.title == "Test"
30 assert result.status == TaskStatus.TODO
31 mock_task_repository.create.assert_called_once()
32
```

Naming Conventions & Best Practices

**Entities**`task.py`*Singular nouns***Use Cases**`create_task.py, get_tasks.py`*Verb + noun***Repositories**`task_repository.py`*Noun + _repository***DTOs**`task_dto.py`*Noun + _dto***Interfaces**Prefix with I: `ITaskRepository`*Interface marker***Exceptions**`TaskNotFoundException`*Descriptive +
Exception***Value Objects**`task_status.py, priority.py`*Concept name*

Common Mistakes to Avoid



- ⚠ Using ORM models directly in domain layer
- ⚠ Importing FastAPI or Pydantic in domain/application layers
- ⚠ Skipping the repository pattern and using database directly in use cases
- ⚠ Mixing DTOs with domain entities (they serve different purposes!)
- ⚠ Not using dependency injection (hardcoding dependencies)
- ⚠ Placing business logic in controllers instead of use cases
- ⚠ Forgetting type annotations on methods and fields
- ⚠ Creating circular dependencies between layers

Key Takeaways



- 1 Keep domain layer pure - no external dependencies
- 2 Use interfaces (DIP) to invert dependencies
- 3 Separate concerns: entities $\neq$ DTOs $\neq$ ORM models
- 4 Test business logic without infrastructure
- 5 Use dependency injection to wire components
- 6 Type annotations everywhere for clarity
- 7 Follow naming conventions consistently

Textbooks

- Robert C. Martin, “Clean Architecture: A Craftsman’s Guide to Software Structure and Design,” Prentice Hall, 2017.



Thank You!

Questions?

Start writing clean, maintainable code today!